

MODUL 5 REAL-TIME OPERATING SYSTEM

Abyan Devadi (H1H024049)

Asisten: Athallah Tsany Satriyaji

Tanggal Percobaan: Rabu, 6 Mei 2026

TK244005-Praktikum Sistem Mikrokontroler

Laboratorium Pemrograman – Jurusan Informatika UNSOED

Abstrak

RTOS (Real-Time Operating System) adalah sistem operasi yang dirancang untuk memproses data secara deterministik dan tepat waktu pada perangkat embedded. Praktikum ini mengimplementasikan FreeRTOS pada Arduino Uno melalui dua percobaan: percobaan pertama menjalankan beberapa task secara konkuren dengan `xTaskCreate()` untuk mengamati cara scheduler mengatur eksekusi masing-masing task, sedangkan percobaan kedua menggunakan queue melalui `xQueueCreate()`, `xQueueSend()`, dan `xQueueReceive()` untuk komunikasi antar task. Hasilnya menunjukkan FreeRTOS dapat mengelola banyak task sekaligus dan memungkinkan pertukaran data antar task secara aman. Pemahaman atas kedua konsep ini diperlukan sebelum merancang sistem embedded yang lebih kompleks.

Kata Kunci: RTOS, FreeRTOS, Multitasking, Task Queue, Scheduler, Embedded System, Arduino, Sistem Real-Time

1. PENDAHULUAN

Sistem embedded sudah ada di hampir semua bidang kehidupan modern, dari perangkat rumah tangga pintar dan sistem navigasi hingga alat medis dan kontrol industri. Seiring fungsinya yang makin kompleks, kebutuhan akan pengelolaan sumber daya yang efisien dan respons waktu nyata menjadi semakin kritis. Ada dua pendekatan utama dalam pemrograman embedded: sequential loop konvensional dan berbasis Real-Time Operating System (RTOS).

Pada pemrograman embedded konvensional, seluruh logika dieksekusi secara berurutan di dalam satu loop utama tanpa mekanisme penjadwalan. Pendekatan ini sederhana, tetapi punya keterbatasan nyata ketika sistem harus menangani beberapa pekerjaan sekaligus. Membaca sensor, mengontrol aktuator, mengelola komunikasi serial, dan memperbarui tampilan dalam satu loop adalah hal yang sulit dilakukan dengan tepat, karena satu tugas yang lambat akan menunda semua tugas lainnya.

RTOS menjawab masalah ini secara langsung. Dengan menjalankan beberapa task secara konkuren lewat penjadwalan berbasis prioritas, RTOS membuat sistem embedded seolah memproses banyak pekerjaan secara paralel.

FreeRTOS, salah satu implementasi RTOS open source yang paling banyak digunakan, menjadi pilihan populer pada berbagai mikrokontroler termasuk Arduino karena mudah digunakan dan memiliki dukungan komunitas yang besar.

Praktikum ini memberikan pengalaman langsung mengimplementasikan dua konsep dasar RTOS dengan FreeRTOS pada Arduino Uno. Melalui Percobaan 5A, mahasiswa menjalankan beberapa task konkuren dan mengamati cara scheduler mengatur eksekusi berdasarkan prioritas dan delay. Melalui Percobaan 5B, mahasiswa mempelajari komunikasi antar task menggunakan queue, yang memungkinkan pertukaran data antara task produsen dan konsumen tanpa risiko konflik atau kehilangan data.

2. STUDI PUSTAKA

Pada bagian ini dijelaskan beberapa konsep dasar yang menjadi landasan pemahaman terhadap materi yang dibahas, mencakup protokol komunikasi pada sistem tertanam secara umum, serta penjelasan spesifik mengenai UART, I2C, dan SPI.

2.1 KONSEP DASAR REAL-TIME OPERATING SYSTEM (RTOS)

RTOS adalah sistem operasi yang dirancang untuk aplikasi real-time, yaitu sistem yang dapat memproses data dan merespons dalam batas waktu tertentu secara deterministik. Berbeda dengan sistem operasi umum seperti Windows atau Linux yang mengutamakan throughput dan antarmuka pengguna, RTOS mengutamakan ketepatan waktu, keandalan eksekusi, dan konsistensi respons terhadap event. Yang terpenting dari sebuah RTOS bukan kecepatan, melainkan jaminan bahwa setiap permintaan dan respons akan dilaksanakan dalam periode waktu yang telah ditentukan, apa pun kondisi sistem [3].

Berdasarkan kekakuan dalam memenuhi deadline, RTOS dibagi menjadi dua kategori. Hard RTOS selalu menyelesaikan setiap task sebelum deadline; kegagalan memenuhi deadline bisa merusak sistem secara keseluruhan, sehingga jenis ini digunakan pada aplikasi kritis seperti sistem kendali pesawat,

perangkat medis, dan kontrol industri. Soft RTOS hampir selalu tepat waktu tetapi masih bisa mentoleransi keterlambatan sesekali; dampaknya berupa penurunan performa, bukan kegagalan fatal [1]. Memahami perbedaan ini penting untuk memilih jenis RTOS yang sesuai dengan tingkat kekritisannya suatu aplikasi.

2.2 KOMPONEN DAN ARSITEKTUR INTERNAL RTOS

Sebuah RTOS terdiri dari beberapa komponen utama yang bekerja bersama untuk menghasilkan eksekusi program yang teratur dan dapat diprediksi [3]. Komponen pertama adalah scheduler, elemen inti yang menentukan urutan eksekusi task berdasarkan prioritas yang ditetapkan pengembang. Scheduler preemptive, yang paling umum digunakan pada RTOS modern, segera mengalihkan CPU ke task berprioritas lebih tinggi yang masuk ke state Ready, meski task berprioritas lebih rendah sedang berjalan; ini menghasilkan waktu respons yang optimal dan deterministik [2].

Komponen kedua adalah task itu sendiri, unit program mandiri yang dieksekusi oleh scheduler. Setiap task bisa berada dalam tiga keadaan: Running, saat task berprioritas tertinggi aktif menggunakan CPU; Ready, saat task siap dieksekusi tetapi CPU sedang dipakai task lain yang lebih tinggi prioritasnya; dan Blocked, saat task menunggu suatu event atau data sebelum bisa melanjutkan eksekusi [2]. Komponen ketiga adalah kernel, yang mengelola seluruh task, mengatur komunikasi antar task, serta mengalokasikan memori agar sistem tidak crash atau berperilaku non-deterministik [3].

2.3 MEKANISME MULTITASKING DAN PENJADWALAN TASK

Multitasking pada RTOS memungkinkan beberapa task berjalan secara konkuren melalui penjadwalan berbasis prioritas. Pada preemptive scheduling, task dengan prioritas tertinggi dalam state Ready langsung menghentikan task yang sedang berjalan dan mengambil alih CPU. Cara menentukan prioritas yang tepat adalah dengan mempertimbangkan seberapa sering task perlu dieksekusi; semakin tinggi frekuensinya, semakin tinggi prioritas yang sebaiknya diberikan [2].

Selain penjadwalan berbasis prioritas, RTOS juga mendukung round-robin scheduling untuk task dengan prioritas yang sama, sehingga tidak ada task yang "kelaparan" dari CPU. Mekanisme clock tick, interupsi periodik yang menjadi detak jantung sistem RTOS, menjadi dasar pengukuran dan penjadwalan waktu eksekusi setiap task [2].

Interval clock tick yang lebih kecil menghasilkan resolusi penjadwalan yang lebih halus, tetapi dengan overhead CPU yang lebih besar.

2.4 KOMUNIKASI ANTAR TASK: QUEUE DAN SEMAPHORE

Ketika beberapa task berjalan secara konkuren dan perlu berbagi data atau sumber daya, diperlukan mekanisme komunikasi yang aman untuk mencegah konflik. RTOS menyediakan beberapa primitif sinkronisasi untuk tujuan ini. Semaphore bekerja seperti sinyal lalu lintas yang mengatur akses eksklusif ke shared resource: hanya satu task yang bisa memegangnya pada satu waktu, dan task lain harus menunggu sampai semaphore dilepaskan; mekanisme ini bekerja secara FIFO (First In First Out) [2].

Queue adalah mekanisme komunikasi yang memungkinkan satu task mengirim data terstruktur ke task lain melalui buffer internal yang dikelola kernel. Task pengirim memakai `xQueueSend()` untuk menaruh data ke antrian, sementara task penerima memakai `xQueueReceive()` untuk mengambilnya secara berurutan. Dengan queue, task yang memproduksi data dan yang mengkonsumsinya bisa dipisah dengan bersih, sehingga masing-masing task bisa fokus pada tanggung jawabnya sendiri tanpa mengganggu waktu eksekusi task lain.

2.5 PERFORMA RTOS DIBANDINGKAN SISTEM NON-RTOS

Evaluasi performa RTOS mencakup beberapa metrik utama, di antaranya waktu respons rata-rata, throughput, tingkat error, dan jitter. Penelitian komparatif antara sistem berbasis RTOS dan sistem tanpa RTOS menunjukkan perbedaan yang cukup nyata: sistem dengan RTOS memiliki waktu respons rata-rata lebih rendah, tingkat error lebih kecil, dan utilisasi CPU lebih efisien dibanding sistem konvensional [5]. Ini terjadi karena RTOS mengatur prioritas dan waktu eksekusi setiap proses secara terkoordinasi, bukan membiarkan masing-masing proses berjalan sendiri-sendiri.

Perlu diperhatikan bahwa performa RTOS juga bergantung pada jumlah task yang dibebankan. Pada beban task yang ringan, sistem berbasis native interrupt bisa punya latensi lebih rendah dari RTOS karena overhead manajemen task tidak sebanding dengan sedikitnya pekerjaan yang ada. Sebaliknya, ketika sistem dibebani banyak task sekaligus, RTOS memberikan performa yang jauh lebih baik dan konsisten karena kemampuannya mengelola sumber daya secara terkoordinasi [4]. Trade-off ini penting dipahami untuk memutuskan

kapan RTOS benar-benar menguntungkan dibanding pendekatan yang lebih sederhana.

3. METODOLOGI

3.1 ALAT DAN BAHAN

Percobaan ini menggunakan komponen hardware dan software yang saling mendukung. Di sisi hardware, komponen utama pada kedua percobaan adalah satu unit Arduino Uno. Untuk Percobaan 5A (Multitasking), diperlukan dua buah LED (merah dan kuning) yang masing-masing dihubungkan melalui resistor 220 Ohm hingga 1 kOhm sebagai pembatas arus. Breadboard digunakan sebagai media perakitan tanpa solder, dan kabel jumper untuk menghubungkan antar komponen. Untuk Percobaan 5B (Komunikasi Task), tidak ada komponen tambahan selain Arduino Uno dan kabel USB, karena percobaan ini sepenuhnya menggunakan Serial Monitor sebagai output.

Di sisi software, seluruh percobaan dikerjakan di Arduino IDE versi terbaru dengan library `Arduino_FreeRTOS.h` sebagai pustaka utama RTOS, serta library `queue.h` yang khusus diperlukan pada Percobaan 5B. Serial Monitor bawaan Arduino IDE digunakan untuk memantau output secara real-time pada kedua percobaan.

3.2 LANGKAH KERJA

Percobaan 5A: Percobaan dimulai dengan memastikan Arduino Uno terhubung ke komputer melalui kabel USB. Program ditulis di Arduino IDE dengan menyertakan header `Arduino_FreeRTOS.h`, lalu tiga prototipe fungsi task (`TaskBlink1`, `TaskBlink2`, dan `Taskprint`) dideklarasikan sebelum `setup()`. Di dalam `setup()`, komunikasi serial diinisialisasi pada baud rate 9600, lalu ketiga task didaftarkan ke scheduler menggunakan `xTaskCreate()` dengan stack 128 word, prioritas 1, dan handle NULL. `TaskBlink1` mengontrol LED di pin 8 dengan periode 200 ms, `TaskBlink2` mengontrol LED di pin 7 dengan periode 300 ms, dan `Taskprint` mencetak nilai counter yang bertambah setiap 500 ms ke Serial Monitor. Setelah semua task terdaftar, `vTaskStartScheduler()` dipanggil untuk menyerahkan kendali ke RTOS, sementara `loop()` dibiarkan kosong. Sketch disimpan dengan nama `modul5_multitasking`.

Rangkaian hardware dirakit di breadboard: LED merah dari pin D10 melalui resistor ke GND, dan LED kuning dari pin D8 melalui resistor ke GND yang sama. Program dikompilasi dan diupload ke Arduino Uno, lalu hasilnya diamati dari pola nyala LED dan output Serial Monitor untuk

memverifikasi bahwa ketiga task berjalan dan scheduler berfungsi benar.

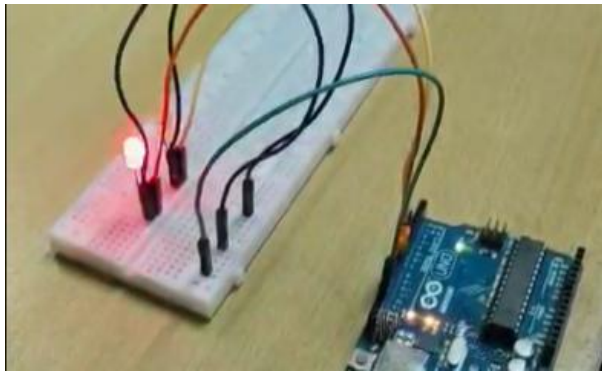
Percobaan 5B: Percobaan ini hanya memerlukan Arduino Uno dan kabel USB. Program ditulis dengan menyertakan `Arduino_FreeRTOS.h` dan `queue.h`, lalu sebuah struct bernama `readings` didefinisikan dengan dua field integer: `temp` dan `h`. Variabel global `QueueHandle_t` bernama `my_queue` dideklarasikan untuk menyimpan referensi queue. Di dalam `setup()`, komunikasi serial diinisialisasi pada baud rate 9600, lalu queue dibuat dengan `xQueueCreate()` berkapasitas satu elemen bertipe struct `readings`. Dua task kemudian didaftarkan: `read_data` dengan prioritas 0, yang mengisi struct dengan nilai statis (`temp = 54`, `h = 30`) lalu mengirimnya ke queue setiap 100 ms menggunakan `xQueueSend()`; serta `display` dengan prioritas 0, yang memblokir diri menunggu data dari queue menggunakan `xQueueReceive()` dengan timeout `portMAX_DELAY`, kemudian menampilkan nilai `temp` dan `h` ke Serial Monitor. `Loop()` dibiarkan kosong. Sketch disimpan dengan nama `modul6_taskqueue`.

Program dikompilasi dan diupload ke Arduino Uno, lalu Serial Monitor dibuka untuk mengamati apakah nilai suhu dan kelembapan tampil berulang secara konsisten sebagai bukti queue bekerja benar.

4. HASIL DAN ANALISIS

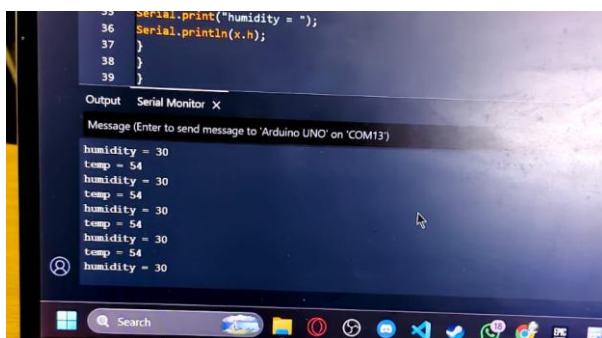
4.1 HASIL PERCOBAAN

Percobaan modul 5 dilaksanakan dalam dua skenario implementasi FreeRTOS pada Arduino Uno. Pada Percobaan 5A, tiga task independen didaftarkan ke scheduler melalui `xTaskCreate()` di dalam `setup()`. Task pertama, `TaskBlink1`, mengendalikan LED di pin 8 dengan siklus nyala-mati 200 ms menggunakan `vTaskDelay(200 / portTICK_PERIOD_MS)`, sekaligus mencetak "Task1" ke Serial Monitor setiap siklus. Task kedua, `TaskBlink2`, bekerja serupa pada LED di pin 7 dengan interval 300 ms dan mencetak "Task2". Task ketiga, `Taskprint`, menginkrement counter setiap 500 ms dan mencetak nilainya ke Serial Monitor. Ketiga task didaftarkan dengan prioritas 1 dan stack 128 word. Setelah `vTaskStartScheduler()` dipanggil, output dari ketiga task muncul bergantian di Serial Monitor tanpa saling memblokir, yang membuktikan context switching berjalan normal. LED di pin 8 dan pin 7 berkedip bersamaan dengan frekuensi berbeda, mengonfirmasi kedua task berjalan secara konkuren.



Gambar 1. Percobaan 5A

Pada Percobaan 5B, queue berkapasitas satu elemen dibuat dengan `xQueueCreate(1, sizeof(struct readings))`, di mana setiap elemen bertipe `struct readings` dengan field `temp` dan `h`. Task `read_data` sebagai producer mengisi `struct` dengan nilai 54 dan 30, mengirimkannya ke queue melalui `xQueueSend()` dengan `portMAX_DELAY`, lalu menunggu 100 tick. Task `display` sebagai consumer menunggu data dari queue dengan `xQueueReceive()` dan `portMAX_DELAY`, lalu mencetak nilai `temp` dan `h` ke Serial Monitor saat penerimaan berhasil (`pdPASS`). Hasilnya, Serial Monitor menampilkan "temp = 54" dan "humidity = 30" secara berulang dan konsisten, yang membuktikan sinkronisasi data antar task melalui queue berjalan tanpa race condition.



Gambar 2. Percobaan 3B

4.2 PERBEDAAN FUNGSI LOOP() PADA ARDUINO BIASA DAN SISTEM BERBASIS RTOS

Pada pemrograman Arduino konvensional, fungsi `loop()` adalah inti dari seluruh logika program. Mikrokontroler mengeksekusi instruksi di dalamnya secara sekuensial dan berulang dalam satu thread tunggal. Jika ada dua tugas yang perlu berjalan bersamaan, misalnya mengedipkan LED dengan interval berbeda sambil mencetak data ke serial, programmer harus mengatur timing-nya secara manual menggunakan `millis()` atau state machine yang kompleks. Semua beban koordinasi itu ada di tangan programmer.

Pada sistem FreeRTOS, cara kerjanya berbeda. Setiap tugas yang perlu berjalan secara konkuren didefinisikan sebagai task tersendiri melalui `xTaskCreate()`, dengan stack, konteks eksekusi, dan prioritasnya masing-masing. Scheduler mengelola kapan setiap task mendapat giliran CPU lewat preemptive scheduling. Ketika sebuah task memanggil `vTaskDelay()`, task tersebut masuk ke state blocked dan CPU langsung dialihkan ke task lain yang siap berjalan. Dengan cara ini, `TaskBlink1`, `TaskBlink2`, dan `Taskprint` pada Percobaan 5A bisa berjalan secara konkuren meski hanya ada satu inti prosesor. Singkatnya: `loop()` pada Arduino biasa harus memuat seluruh logika program, sedangkan pada RTOS, `loop()` tidak lagi relevan karena scheduler yang memegang kendali.

4.3 ALASAN FUNGSI LOOP() DIBIARKAN KOSONG

`Loop()` dibiarkan kosong karena kendali eksekusi sudah diserahkan ke RTOS scheduler lewat pemanggilan `vTaskStartScheduler()` di akhir setup(). Setelah fungsi itu dipanggil, scheduler mengambil alih CPU dan mulai menjadwalkan task yang sudah terdaftar. Secara teknis, `vTaskStartScheduler()` tidak pernah mengembalikan nilai ke titik pemanggilan selama sistem RTOS berjalan normal, jadi eksekusi program tidak pernah mencapai baris setelahnya, termasuk `loop()`.

Jika ada instruksi di dalam `loop()`, kode tersebut pada dasarnya tidak akan pernah dijalankan selama scheduler berjalan benar. Menempatkan logika di `loop()` dalam konteks RTOS tidak hanya tidak perlu, tetapi juga berpotensi membingungkan arsitektur program. Pada kedua percobaan ini, seluruh logika program (kendali LED, pencetakan serial, penghitungan counter, komunikasi queue) dideklarasikan di dalam fungsi task masing-masing dan didaftarkan sebelum scheduler dijalankan. `Loop()` yang kosong bukan kelalaian, melainkan keputusan desain yang mencerminkan bahwa tanggung jawab eksekusi telah berpindah dari model loop tunggal ke model multitasking berbasis scheduler.

4.4 INSIGHT UTAMA DARI PRAKTIKUM

Praktikum ini memperlihatkan bahwa FreeRTOS bukan sekadar library tambahan. Ia mengubah cara berpikir tentang eksekusi program pada mikrokontroler. Hal yang paling mencolok adalah konkurensi bisa dicapai pada hardware berinti tunggal bukan melalui pemrosesan paralel sejati, melainkan lewat pembagian waktu CPU yang sangat cepat oleh scheduler, sehingga setiap task seolah berjalan bersamaan. Ini terbukti pada

Percobaan 5A: LED di pin 7 dan pin 8 berkedip bersamaan dengan frekuensi berbeda meski keduanya dieksekusi oleh satu prosesor yang sama.

Percobaan 5B menunjukkan pentingnya sinkronisasi antar task. Tanpa mekanisme seperti queue, dua task yang berbagi data yang sama rentan terhadap race condition: satu task membaca data yang sedang ditulis task lain, menghasilkan nilai yang tidak konsisten. Penggunaan `xQueueSend()` dan `xQueueReceive()` dengan `portMAX_DELAY` menjamin urutan eksekusi antara producer dan consumer: consumer hanya memproses data setelah producer benar-benar selesai mengirimnya. Mekanisme blocking ini juga menghilangkan kebutuhan polling aktif yang memboroskan CPU. Secara keseluruhan, praktikum ini menegaskan bahwa RTOS adalah kompetensi yang kritis untuk membangun sistem embedded yang real-time, modular, dan handal.

5. KESIMPULAN

Praktikum modul 5 membuktikan bahwa FreeRTOS memungkinkan implementasi multitasking yang terstruktur pada Arduino Uno. Pada Percobaan 5A, tiga task konkuren dengan interval berbeda berjalan bersamaan tanpa saling mengganggu berkat preemptive scheduling dan `vTaskDelay()` yang memungkinkan task melepaskan CPU saat tidak aktif. Pada Percobaan 5B, mekanisme queue terbukti efektif sebagai sarana komunikasi antar task yang aman, memastikan transfer data antara producer dan consumer berlangsung sinkron dan bebas dari race condition. Memindahkan seluruh logika dari `loop()` ke dalam task RTOS adalah perubahan cara berpikir yang nyata: dari eksekusi sekuensial menjadi multitasking berbasis scheduler. Pemahaman atas konsep ini, mencakup manajemen task, sinkronisasi, dan mekanisme blocking, menjadi dasar yang diperlukan untuk merancang sistem embedded berskala lebih besar yang bersifat real-time.

DAFTAR PUSTAKA

- [1] R. Setiaji, M. Syahrul, A. Fajar, M. Ihsan, dan F. Sinlae, *Pengembangan Sistem Operasi Real-time Untuk Aplikasi Embedded*, ARembeN: Jurnal Pengabdian Multidisiplin, vol. 2, no. 1, hal. 18–23, Juni 2024.
- [2] B. P. Leksono, I. Setiawan, dan B. Setiyono, *Penerapan Real Time Operating Systems (RTOS) Pada Mikrokontroler AVR (Studi Kasus ChibiOS/RT)*, Makalah Seminar Tugas Akhir, Jurusan Teknik Elektro, Fakultas Teknik, Universitas Diponegoro, Semarang.
- [3] E. S. Manapa, E. A. M. Sampetoding, T. W. Sagala, dan H. R. Taluay, *Ulasan Penelitian Sistem Operasi Waktu Nyata di Indonesia*, Prosiding Seminar Nasional, hal. 973–979, 2021.
- [4] N. Habibie, M. R. Alhamidi, D. M. J. Purnomo, dan M. F. Rachmadi, *Performance Comparison of USART Communication Between Real Time Operating System and Native Interrupt*, Jurnal Ilmu Komputer dan Informasi, vol. 9, no. 1, hal. 43–51, Februari 2016.
- [5] V. Veliza dan R. Saputra, *Analisa Sistem Operasi Real-Time pada Pengendalian Robot Industri: Evaluasi Performa dan Stabilitas*, Polygon: Jurnal Ilmu Komputer dan Ilmu Pengetahuan Alam, vol. 3, no. 6, hal. 60–72, November 2025.